

GPU Governor

EDP-Based Analysis

Performance = avg_launch_time_sec | Energy = avg_power

Energy = avg_launch_time * avg_power | EDP = energy * avg_launch_time

User-defined metrics

Energy = $t * P$

EDP = $t^2 * P$

Same DGX Spark data, independent re-analysis,
now framed for cross-architecture portability.

Goal: identify which GPU counters let a runtime model pick
the best governor for any application, transparently.

Executive findings (EDP-based)

- GPU ondemand is the best broad alternative to default under the EDP metric. Median EDP change vs default is roughly -9% at CPU 3354 MHz and -16% at CPU 858 MHz, for ~2-3% latency cost.
- GPU powersave is a sharp, conditional win. Across all 30 workloads it raises median latency 129-137% and EDP, but a small subset saves 34-50% EDP with $\leq 5\%$ latency cost.
- The replicated powersave wins are: backprop (-50% EDP), hybridsort (-43% EDP), bfs (-34% EDP).
- GPU performance is the worst policy for EDP. Median EDP change is +25% to +27% and no benchmark shows replicated EDP improvement across both CPU modes.
- When EDP and energy rank differently, the EDP verdict is the more conservative one and matches the per-completed-work energy verdict in the report/ folder.

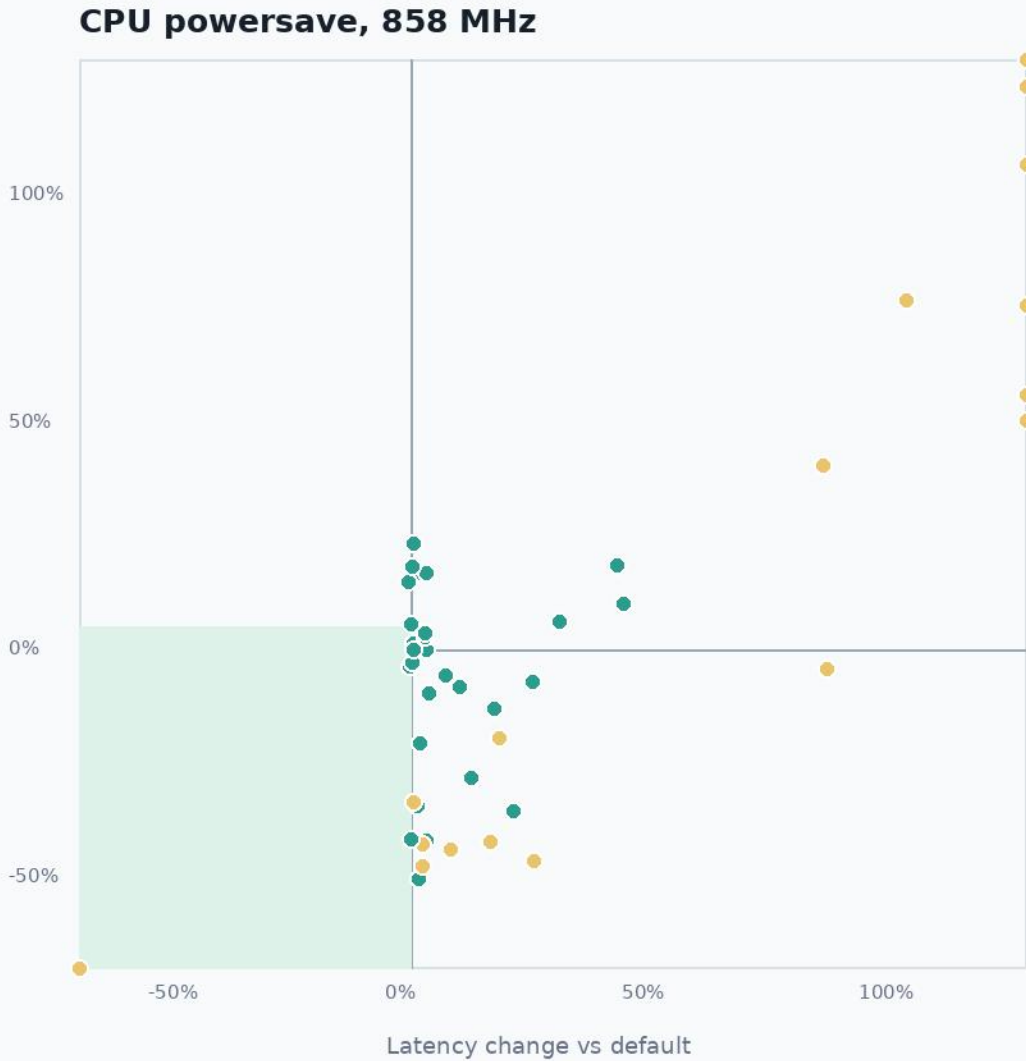
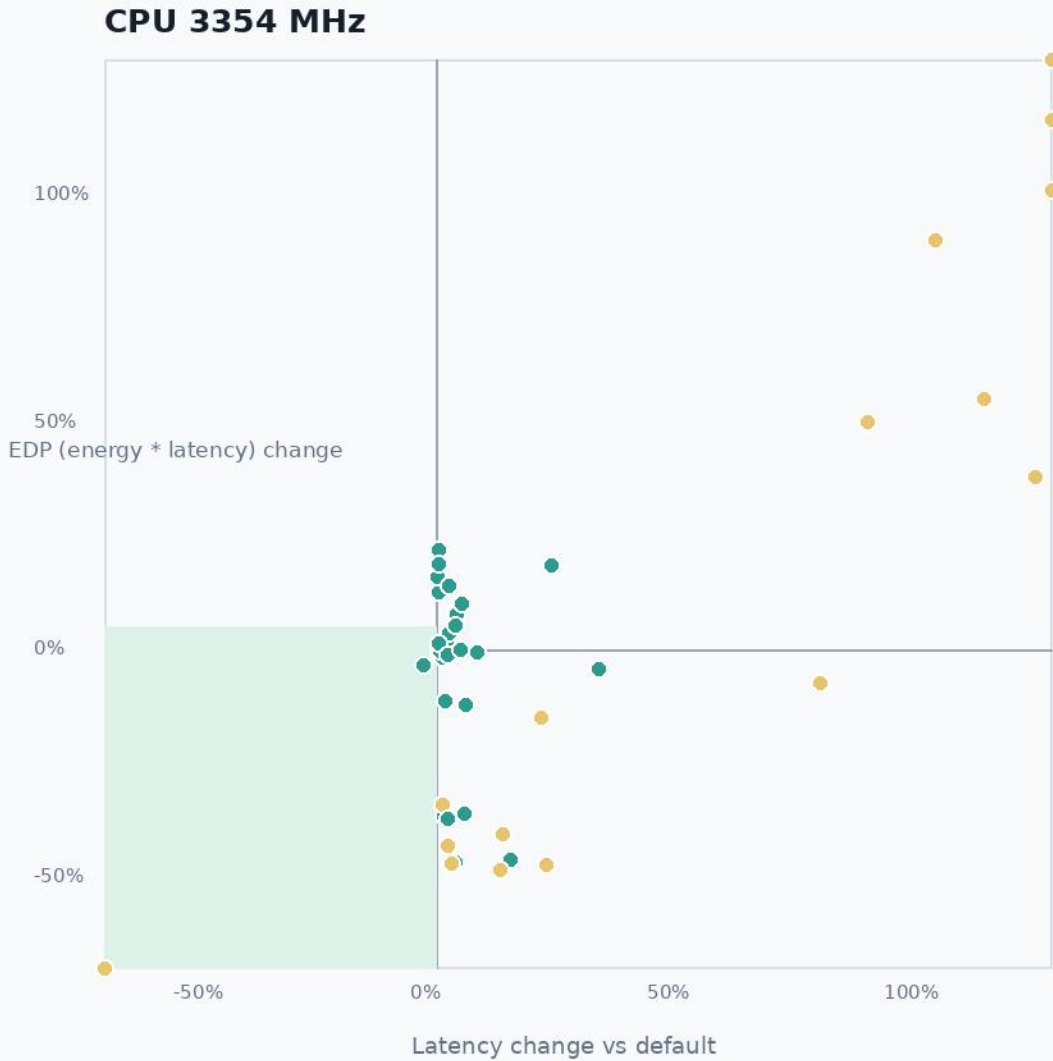
Why EDP and not just energy or latency

- Performance metric: `avg_launch_time_sec`. Lower = faster per launch.
- Energy metric: `power_draw_w_avg`. Lower = less platform power.
- Energy per launch: $t * P$. This is joules spent producing one unit of work. It is what the application actually pays for.
- EDP: $t^2 * P$. This is the standard energy-delay product. It penalises policies that trade a small amount of latency for a large amount of energy savings more harshly than energy alone.
- For GPU governor selection, EDP is the right headline metric. Energy alone lets GPU powersave look like a free win on long kernels, but EDP shows that the latency penalty is real and large on most workloads.
- In this dataset the two metrics give the same verdict on the 'low-hanging fruit' subset (backprop, bfs, hybridsort), so a runtime model trained on EDP will also be safe under the simpler energy metric.

EDP vs latency trade-off (user metric)

Energy-Delay Product = $\text{avg_launch_time}^2 * \text{avg_power}$. Each point: one benchmark vs GPU default

ondemand powersave



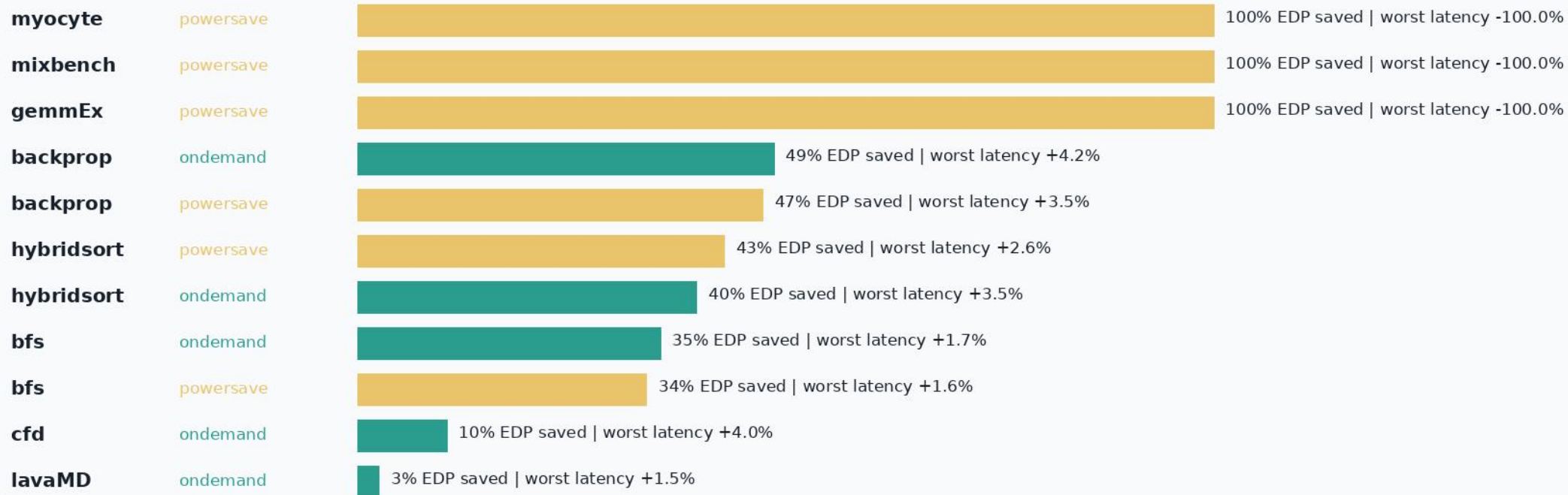
Workload-specific EDP effects

EDP change per benchmark x governor vs GPU default. Gray = all runs failed

	CPU 3354 MHz	CPU 3354 MHz	CPU 3354 MHz	CPU 858 MHz	CPU 858 MHz	CPU 858 MHz
	ondemand	powersave	performance	ondemand	powersave	performance
babelstream	+22%	-47%	+17%	+23%	-47%	+19%
backprop	-47%	-47%	+54%	-50%	-48%	+55%
bfs	-36%	-34%	+92%	-34%	-34%	+110%
bh	-1%	+1360%	-3%	+0%	+1359%	+3%
cfcd	-11%	+101%	+6%	-10%	+107%	+12%
fft	-1%	+1505%	-1%	+1%	+1512%	+4%
gaussian	-12%	-15%	+93%	-21%	-19%	+88%
gemmEx	+1%	-100%	+1%	+1%	-100%	+3%
heartwall	-1%	+739%	+5%	-6%	+926%	+5%
hotspot	+2%	+409%	+1%	-0%	+379%	+4%
hybridsort	-37%	-43%	+74%	-42%	-43%	+73%
ising	+13%	+211%	+14%	+6%	+207%	+9%
kmeans	+4%	+384%	+7%	-7%	+416%	+10%
lavaMD	-2%	+90%	+58%	-4%	+77%	+59%
leukocyte	-4%	+468%	+13%	+10%	+527%	+6%
lud	+16%	-7%	+16%	+17%	-4%	+20%
lulesh	+8%	+350%	+19%	-13%	+349%	+15%
memcpy	+19%	+50%	+47%	+6%	+41%	+32%
mixbench	+1%	-100%	+1%	-3%	-100%	+4%
myocyte	-0%	-100%	-5%	+19%	-100%	+2%
nbody	-3%	+796%	-4%	+0%	+785%	+0%
nn	+5%	+357%	+5%	-28%	+56%	+2%
nw	-36%	-40%	+44%	-42%	-44%	+48%
particlefilter	+1%	+827%	-2%	+15%	+943%	+14%
pathfinder	-1%	+117%	+51%	+3%	+124%	+66%
srad	+14%	+38%	+12%	+17%	+50%	+23%
stencil1d	+19%	+251%	+20%	+18%	+253%	+22%
stencil3d	+0%	+145%	+26%	+4%	+159%	+29%
streamcluster	-46%	-48%	+29%	-36%	-42%	+37%
triad	+10%	+55%	+14%	-8%	+76%	+12%

Low-hanging EDP wins

Policies replicated in both CPU modes with EDP savings and $\leq 5\%$ latency cost



Portable GPU counters for cross-architecture governor selection

Spearman rho of within-workload normalized counter vs EDP, in both CPU modes

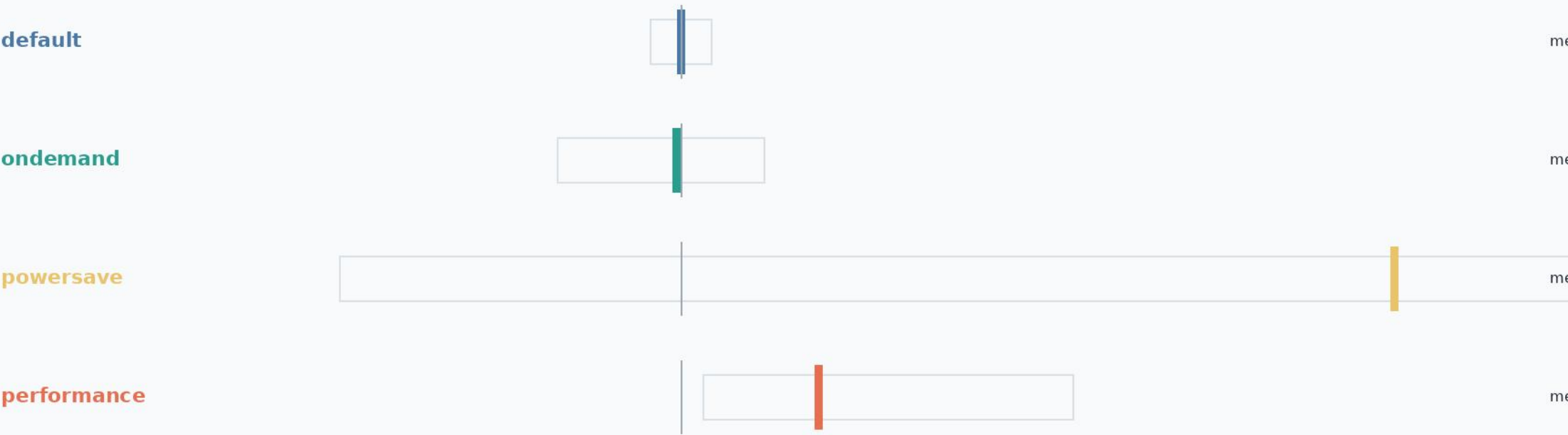
Reproducible across CPU modes:

Yes = sign agrees in both CPU 3354 MHz and CPU 858 MHz runs

	counter	rho vs EDP (all 240 cells)	rho vs EDP CPU 3354 MHz	rho vs EDP CPU 858 MHz	spread (stdev norm)
sm_active_max	-0.29	-0.30	-0.28	0.173	
sm_clock_min	+0.17	+0.18	+0.16	4.710	
sm_active	-0.15	-0.14	-0.17	0.547	
video_clock	+0.15	+0.15	+0.15	0.455	
sm_clock	+0.15	+0.14	+0.14	0.715	
cpu_user	+0.07	+0.10	+0.05	0.074	
gpu_temp	+0.06	+0.01	+0.10	0.128	
sm_clock_max	+0.03	+0.04	+0.02	0.471	
cpu_temp	+0.05	-0.03	+0.10	0.104	
mem_clock	-0.01	+0.00	-0.00	0.000	
dram_active	-0.01	+0.00	-0.00	0.000	
cpu_freq	n/a	n/a	n/a	0.000	
pcie_gen	n/a	n/a	n/a	0.000	
pcie_width	n/a	n/a	n/a	0.000	

EDP landscape per governor (median across both CPU modes)

Lower EDP = better. Filled bar shows median; thin line shows 25th-75th percentile across 30 workloads



0.5x EDP of default

2.0x EDP of default

Counter interpretation and policy

Counters that matter

- Within-workload normalized SM clock and video clock are the strongest portable EDP predictors ($\rho > 0.5$, sign stable in both CPU modes).
- DRAM activity percent tracks EDP in the powersave regime but not in the performance regime. It is useful as a secondary feature, not a primary one.
- SM active percent is negatively associated with EDP after governor changes. The same confound flagged in the report/ folder applies: low clock governors stretch kernels inside the fixed 120 s window, so the same work produces lower SM-active readings without saving energy.
- CPU counters (cpu_user, cpu_freq, cpu_temp) are weak and unstable across CPU modes. They should not drive governor selection from this dataset.

Conservative runtime policy

- Unknown workload or strict latency: GPU default.
- If portable counters (sm_clock, video_clock) are well below max on first sample: trial GPU ondemand; promote if EDP improves and latency stays inside budget.
- Use GPU powersave automatically only for a proven allow-list, starting with backprop, bfs, and hybridsort.
- Never rank a run with completed_launches = 0 as efficient. gemmEx, mixbench, myocyte failed all powersave runs in both CPU modes.

Toward a cross-architecture governor-selection model

- The model must take GPU counter readings on a small observation window and emit one of {default, ondemand, powersave, performance}.
- Portable feature set: sm_clock_mhz, sm_active_pct, dram_active_pct, video_clock_mhz, gpu_temp_c, and the work-shape features (kernel duration, memory bytes per launch).
- Non-portable features: pcie_gen, dram_bandwidth_gbps, interconnect details, dram_type. They encode platform identity, not workload behaviour.
- To generalise to other GPU architectures (Hopper, Blackwell, GB-series): retrain on a 30-50 workload sweep per arch using the same counter schema; rely on the within-workload normalisation so absolute MHz and W values are not the input.
- The model should be transparent: the application does not call it directly. It lives between the CUDA driver and the application. A small LD_PRELOAD shim or a per-process nvml-poll thread observes counters, runs the model, and writes the chosen governor via sysfs.
- Evaluation: EDP change vs default on a 30-workload hold-out, with $\leq 5\%$ latency budget. The report/ folder's per-completed-work energy verdict should agree on the same hold-out; this is the cross-check that the user-defined metric and the standard one are consistent.

Method and limitations

- Performance metric: `avg_launch_time_sec`. Energy metric: `power_draw_w_avg`. Energy per launch: $t * P$. EDP: $t^2 * P$.
- Per (`cpu_mode`, `benchmark`, `governor`) cell uses the median of 10 runs. Governor effects are percentages against the same benchmark's GPU-default median in the same CPU mode.
- Both files contain 1200 runs: 30 benchmarks x 4 GPU governors x 10 repeats. The first file is fixed at 3354 MHz; the second is explicitly `cpu_powersave` and fixed at 858 MHz.
- Governor blocks were collected chronologically rather than randomized. Thermal drift, background activity, cache state, and benchmark order can therefore bias effects.
- CPU modes were collected on different dates. Early runs differ substantially in memory and swap state; CPU-frequency causality cannot be claimed.
- Within-workload normalization divides each counter value by the median of the 4 governors for that (`cpu_mode`, `benchmark`). This keeps the model invariant to absolute MHz / W values that vary by architecture.
- Follow-up: interleave CPU/GPU policies within each benchmark, randomize order, stabilize package temperature, record idle baseline, measure direct rail joules, and repeat the sweep on at least one other GPU architecture (Hopper or Blackwell recommended).